HEBCPF Solver Suite

Overview and Version Selection Guide

Finding all *real-valued solutions of the AC power-flow equations*
Ellipsoidal Formulation + Holomorphic Embedding + Padé + Numerical Continuation

This guide compares the **six** packaged releases and helps you pick one.

Release folder	Orientation
`HEBCPF_MEX_v4_20260714`	Performance-optimized compiled build, Windows x64
`HEBCPF_matlab_v4_20260714`	Performance-optimized portable build, any OS
`HEBCPF_MEX_v3.20260712`	Robustness-oriented, compiled — hardened config, Windows x64
`HEBCPF_matlab_v3.20260712`	Robustness-oriented, portable — hardened config, any OS
`HEBCPF_MEX_v2.20260712`	Efficiency-oriented, compiled — legacy-fast config, Windows x64
`HEBCPF_matlab_v2.20260712`	Efficiency-oriented, portable — legacy-fast config, any OS

Solver lines are v4.20260714, v3.202606, and v2.202607; this package release is dated 2026.07.14.

Each release also ships its own detailed `HEBCPF_User_Guide.pdf`. When you publish results, cite the HEBC paper (Section 9).

Package release: July 14, 2026

Contents

1 What all six releases have in common

Every release implements the **same method** — the Holomorphic Embedding Based Continuation (HEBC) all-solutions power-flow tracer — and returns the same solution set on the completed benchmark cases. They also share:

- the same dependency: MATLAB + MATPOWER (`runpf` for the initial point and `makeYbus` for the bus-admittance matrix);
- the same three execution frameworks — serial (`run_batch.m`), `parfor` (`run_batch_par.m`), and `parfeval` (`run_batch_parfeval.m`, recommended for large cases);
- the same single-case programmatic harness `run_merged_case`;
- identical bundled test systems and the same published solution-count references (Table 1);
- the same bordered-Newton formulation on the numerical hot path. MEX releases use the KLU symbolic-once refactorisation, which keeps the *exact* Jacobian and scales to $\sim$7.5$\times$ on `case118`. MATLAB v2/v3 can use a compatible `klurf` MEX; MATLAB v4 deliberately uses MATLAB built-ins only.

On completed benchmark cases, **the choice between releases is about *robustness*, *speed*, and *platform* — never about correctness of the answer on well-behaved systems.**

Table 1: Reference number of real solutions (antipodal pairs collapsed). Values through `case57` are completed benchmarks; `case118` is a partial enumeration, not a claimed total.

System	# sol.	System	# sol.
`case3`	6	`case14mod`	30
`case4gs`	6	`case33bw`	16
`case4BB0`	14	`case39`	176
`case4BBc`	12	`case30/case_ieee30`	472
`case5loop`	10	`case57mod`	606
`case5Salam`	10	`case57`	1322
`case6ww`	6	`case118`	$> 1.5 \times 10^6$ (partial)
`case7Salam`	4		
`case9/case9Q`	8		

2 Changes in package release 2026.07.14

Compared with the publicly released `HEBCPF` baseline folders (solver lines `v2.202607` and `v3.202606`), this package keeps the HEBC formulation and the v2/v3 numerical configurations unchanged. Package 2026.07.12 added:

- the bordered Newton corrector now has a sparse-core Schur-complement path with SuiteSparse/KLU symbolic-once refactorisation. The MEX releases include `klurf.mexw64`; the top-level `klurf/` folder provides the source and build helper for other MEX platforms or for the optional-KLU MATLAB v2/v3 releases;
- keyed bucket lookup accelerates repeated-solution collection in the serial, `parfor`, and `parfeval` collectors;
- geometric `Zsave` growth avoids repeated full-array copies while collecting newly discovered solutions;

- the Jacobian cache has a lower-overhead hot path after its per-trace initialisation.

Package 2026.07.14 adds the compiled `HEBCPF_MEX_v4_20260714` release. Relative to MEX v3, v4 uses a global per-equation `parfeval` queue, caches the sparse LU ordering of the holomorphic matrix, reuses sparse holomorphic-assembly invariants, and evaluates Padé polynomials with matrix-vector products instead of replicated temporary matrices. The prior barrier scheduler remains available as `runVBook_hybrid_parfeval_barrier.m` for controlled comparisons. The same release also adds `HEBCPF_matlab_v4_20260714`: the portable counterpart retains those v4 scheduling and holomorphic-kernel optimizations while using MATLAB built-in linear algebra exclusively in its correctors.

3 The two axes of choice

The v2/v3 releases are the combinations of two independent choices. MEX v4 and pure MATLAB v4 are performance-optimized successors to their v3 counterparts and keep the v3 numerical configuration.

Axis 1 — v2 (legacy-fast) vs. v3 (validated / robust)

This is the important one. **v2** is the fast legacy configuration; **v3** is the robustness-hardened configuration developed and validated to eliminate the numerical failures (wrong-branch drift, non-closing loops) that the legacy settings can hit on sharp or fold-dense curves.

Aspect	v2 — legacy-fast	v3 — validated / robust
Settings location	inlined in `hybrid_traceloops`	centralised in `solver_params.m`
Turning-point slope limit (`slope_max`)	`4e5`	`4e5` — shared release default
Hand-back hysteresis (`handback`)	`0.1`	`0.5` — clears the fold first
Fold initial-step cap (`init_cap_k`)	*none*	`1500`
Adaptive step cap (`max_adapt`)	*none*	`true` — secant-slope driven
Padé pole estimate	consensus (`'fixed'`/`'random'` toggle)	consensus, fixed bus set `2:6`
Speed	faster (≈10–20% in parfeval)	modestly slower in parfeval
Robustness	may stall (500-cap) on hard curves	hardened by fold-step and hand-back safeguards

Both give the same counts on well-behaved systems. The difference shows up on *hard* curves (sharp thin-sliver folds, fold-dense loops — e.g. some case118, case30 and case57mod curves): v2 may burn its alternation cap on a wrong/non-closing branch (costing time; the solution is still found by a neighbouring trace so the *count* stays exact), whereas v3 keeps the trace on the true branch by adding the three safeguards below.

The extra conditions and criteria in v3 (absent in v2)

Every safeguard acts *only near singular points* (turning points / folds) and during the hand-off between the holomorphic and continuation routines. With the shared `slope_max = 4e5` default, the three additional v3 safeguards are:

1. **Fold-proximity initial-step cap** (`init_cap_k = 1500`). When the holomorphic routine hands over to the continuation near a fold, the holomorphic arc that just collapsed, `holo_arc`, is a proxy for how close the fold is. v3 caps the *first* continuation step to `astep <= init_cap_k * holo_arc / sstep`: a near-fold hand-off (small `holo_arc`) forces a small, careful first step, while a smooth hand-off (large `holo_arc`) is left at full step. v2 has no such cap, so its first step can be large enough to jump the fold onto the adjacent (wrong) branch.

2. **Secant-slope adaptive per-step cap** (`max_adapt = true`). At every continuation step v3 measures the two-point secant slope $s_k = \|\Delta\text{state}\|/|\Delta\alpha|$ and caps the next step by how fast that slope is changing: `astep <= maximumstep * min(s_prev/s, 1)`. As the curve steepens toward a fold (s rising) the ceiling tightens automatically; once the fold is passed it relaxes back to the full `maximumstep`. v2 has no curvature-aware per-step cap.

3. **Larger hand-back hysteresis** (`handback`: `0.5` in v3 vs `0.1` in v2). Before handing back, the continuation must travel `handback` × `aal_min` *past* the homotopy-parameter (α) reversal, so the near-singular zone is fully cleared. v3 clears five times as much of that zone as v2, which is what stops the curve being pulled back onto an adjacent branch on the holomorphic restart.

Because these fire only near singularities, v3's overall speed cost is small (Section 6) while its robustness gain on hard curves is decisive.

Axis 2 — MEX vs. MATLAB (pure)

Aspect	MEX build	MATLAB (pure) build
Hot path	2 compiled MEX (`Pade_Apprxmt`, `holomorphic_cont_tri`)	plain `.m` only
Bordered solve	KLU symbolic-once refactor — `klurf.mexw64` shipped, on by default	MATLAB v4: built-in \; MATLAB v3/v2: built-in or compatible `klurf`
Serial speed	$\approx 2\times$ **faster**	baseline
Platform	Windows 64-bit (shipped binaries)	**any** — Windows / Linux / macOS
Compiler	MATLAB Coder, only to rebuild or port	none, ever
Extra files	`*.mexw64`, `build_mex.m`, `examples.mat`	*none*
Numerical result	*identical* (verified to $\sim 10^{-13}$)	

The MEX and MATLAB builds of the *same* version are designed to return interchangeable, bit-comparable solution sets; MEX is primarily a speed/platform choice. MATLAB v4 deliberately keeps its corrector path pure MATLAB and never calls `klurf`.

4 Choosing a release

All six releases are intended to return the same solution set. **When no specific compatibility, legacy-reproduction, or platform preference applies, choose v4 as the default solver**: choose **MEX v4** on supported Windows x64 installations, and choose **MATLAB v4** on other platforms or when a pure-MATLAB implementation is preferred. The v3 and v2 releases remain available as established and legacy reference configurations.

MEX v4
Choose this on supported Windows x64 installations when wall-clock performance is the priority. It retains the v3 hardened numerical configuration, adds the global `parfeval` work queue and cached hot-path kernels, and has reported `parfeval` speed-ups of 1.21–1.59× over MEX v3 on six cases (Section 6).

MATLAB v4
Choose this when portability and readable `.m`-only code matter, while still wanting the v4 scheduling and cached numerical kernels. Its correctors use only MATLAB built-ins; no MEX binary, compiler, KLU, or `klurf` dependency is required.

MEX v3 / MATLAB v3
Choose an established hardened v3 release when reproducing earlier v3 work or when the v4 changes are not desired. MEX v3 is the compiled Windows x64 option; MATLAB v3 is the portable option.

MEX v2 / MATLAB v2
Choose only for legacy reproduction or for well-behaved cases where the historical v2 configuration is specifically wanted. It can be faster under historical `parfeval` measurements, but it lacks v3/v4's fold-hardening safeguards.

5 Feature matrix

	MEX_v4	matlab_v4	MEX_v3	matlab_v3	MEX_v2	matlab_v2
Robustness-hardened config (v3)	yes	yes	yes	yes	no	no
`solver_params.m` single-source	yes	yes	yes	yes	no	no
Fold-proximity step caps	yes	yes	yes	yes	no	no
Consensus Padé poles	yes	yes	yes	yes	yes	yes
`'fixed'`/`'random'` pole toggle	no	no	no	no	yes	yes
Global `parfeval` work queue	yes	yes	no	no	no	no
Cached holomorphic sparse-LU ordering	yes	yes	no	no	no	no
Reused sparse holomorphic assembly invariants	yes	yes	no	no	no	no
Allocation-free Padé evaluation	yes	yes	no	no	no	no
Corrector uses MATLAB built-ins only	no	yes	no	yes	no	yes
Compiled MEX (≈ 2× serial)	yes	no	yes	no	yes	no
Cross-platform (no compiler)	no	yes	no	yes	no	yes
Serial / `parfor` / `parfeval`	yes	yes	yes	yes	yes	yes
Identical solution counts	yes	yes	yes	yes	yes	yes

6 Numerical test bed and timing comparison

6.1 Test bed

All timings in this section were measured on a single dedicated workstation, one MATLAB process at a time (no concurrent jobs), to minimise interference:

Component	Specification
CPU	Intel Core i9-13900K — 24 cores / 32 threads (8 performance + 16 efficiency cores)
RAM	128 GB
Operating system	Microsoft Windows 11 Pro, version 22H2 (build 22621)
MATLAB	R2022a (9.12.0), `maxNumCompThreads` = 24
Parallel pool	`local` profile, 23 workers
Initial solve	MATPOWER 7.1 `runpf` (for the starting operating point)

For the completed cases listed in the timing tables, solution *counts* were verified identical across releases; only wall-clock time differs. **Serial** figures are a single timed run per case; **`parfeval`** figures are the *mean of three back-to-back trials* on a warmed pool (the one-time ∼45 s pool start-up is excluded). All times are the trace wall-clock returned by `run_merged_case`.

These figures are a *historical dense-bordered-solve baseline*; they are retained for version selection and are not an end-to-end timing claim for package 2026.07.14. The current MEX releases ship the **KLU symbolic-once refactor**. MATLAB v3 can use a compatible `klurf` MEX, whereas MATLAB v4 deliberately keeps its correctors on MATLAB built-ins. KLU further reduces time on larger systems — a wash below `case30`, but up to ∼7.5× on `case118`. See each User Guide for the applicable linear-solver details.

6.2 All reported MEX v4 `parfeval` timings

MEX v4 was developed against MEX v3 on the same 23-worker local-pool setup. The following development-time values are means of two runs, excluding pool start-up; they are included as a reference and were **not re-executed during this package-preparation pass**. The development record reports order-insensitive matching of the same solution sets through `case57`.

System	MEX v3 (s)	MEX v4 (s)	v3/v4 speed-up
`case9`	1.61	1.33	1.21×
`case14mod`	5.57	3.91	1.42×
`case33bw`	8.22	5.66	1.45×
`case39`	49.93	38.64	1.29×
`case30`	101.81	63.96	1.59×
`case57mod`	236.43	175.70	1.35×

6.3 MATLAB v4 validation results

MATLAB v4 has a documented correctness-validation record, rather than a corresponding wall-clock timing series. The following are all recorded MATLAB v4 results from 2026-07-14; the set errors compare MATLAB v4 with MEX v4 in both matching directions.

Run or comparison	Recorded result
`case3`, serial	6 solutions; maximum residual `1.69e-14`
`case3`, MATLAB v4 vs. MEX v4	6 vs. 6; bidirectional set error `2.42e-14` L1
`case9`, serial	8 solutions; maximum residual `7.39e-13`
`case9`, MATLAB v4 vs. MEX v4	8 vs. 8; bidirectional set error `2.89e-12` L1
`case14mod`, serial	30 solutions; maximum residual `9.25e-13`
`case14mod`, MATLAB v4 vs. MEX v4	30 vs. 30; bidirectional set error `2.49e-12` L1
`case3`, `parfeval`	pool started; 6 solutions; maximum residual `2.35e-14`

No MATLAB v4 wall-clock series was supplied, so this overview makes no numerical timing claim for MATLAB v4 versus MATLAB v3, MEX v4, or the v2 releases. Its v4 queue and cached kernels are an implementation improvement; its all-MATLAB corrector is a deliberate portability trade-off rather than a measured speed conclusion.

6.4 Serial timings — single trial (seconds)

System	# sol.	MEX_v2	MEX_v3	matlab_v2	matlab_v3
case9	8	1.4	1.5	2.0	2.4
case14mod	30	8.2	9.4	14.0	15.9
case33bw	16	19.9	25.1	33.7	42.4
case39	176	513	440	558	627
case30	472	946	527	793	897
case57mod	606	4502	2737	3548	3984

6.5 parfeval timings — mean of 3 trials (seconds)

System	# sol.	MEX_v2	MEX_v3	matlab_v2	matlab_v3
case9	8	0.6	0.7	0.9	1.0
case14mod	30	3.5	3.8	5.8	6.0
case33bw	16	5.6	8.4	10.3	12.7
case39	176	49.3	58.7	71.1	75.2
case30	472	90.6	104.3	169.4	162.3
case57mod	606	268.7	304.9	408.3	421.1

How to read these tables — four findings.

1. **MEX v4 improves on MEX v3 in every recorded v4 timing.** Across all six reported parfeval cases, MEX v4 is 1.21–1.59× faster than MEX v3. This is the directly comparable v4 performance result: both columns use the same development-time two-run, 23-worker protocol.

2. **Use parfeval for anything large.** On this 24-core machine parfeval is **5–17**× faster than serial on the big cases (e.g. MEX_v3 case57mod: 2737 s serial → 305 s parfeval). Serial is best reserved for small systems, reproducibility, and debugging.

3. **The v2-vs-v3 winner depends on the mode.** Under parfeval the legacy *v2* is ≈10–20% faster than the hardened *v3*: its lighter per-step config wins and its occasional wrong-branch stalls are absorbed across 23 workers. In *serial* this *reverses* on the large MEX cases — MEX_v3 beats MEX_v2 by up to **1.6**× (case57mod 2737 s vs 4502 s) — because v2's legacy settings hit wrong-branch stalls that each burn the 500-step cap, and in serial there is no other worker to hide that cost. *So v2's speed edge exists only in parallel; for serial runs on large or hard systems v3 is both safer and faster.* (v2's large-case serial times even differ between the MEX and pure-MATLAB builds because tiny floating-point differences change which curves stall — a further argument for v3's predictability.)

4. **MEX vs pure-MATLAB.** Under parfeval MEX is ≈1.2–1.6× faster than the pure-MATLAB build of the same version (v3 case57mod 305 s vs 421 s). This is smaller than the ≈ 2× MEX advantage seen in *serial*, because parfeval already spreads the work across workers so per-trace speed matters less. This historical observation applies to v2/v3 only; MATLAB v4 has validation results above, but no recorded timing series.

The historical v2/v3 tables and the MEX v3/v4 table use different benchmark passes and trial counts, so they should not be combined into a direct six-release ranking. Times carry the usual parallel run-to-run variance (±10–15%); read each internally consistent table as a ratio guide, not as an absolute cross-table ranking.

7 Where to go next

Each release folder contains its own `HEBCPF_User_Guide.pdf` with installation, quick-start, full parameter reference, the `parfor`-vs-`parfeval` explanation, and troubleshooting specific to that build. Start there once you have picked a release from this guide.

8 License and third-party notices

HEBCPF is distributed under the BSD 3-Clause License; see the top-level `LICENSE` file. The suite depends on MATPOWER at runtime and calls MATPOWER functions including `runpf`, `makeYbus`, `idx_bus`, and `idx_brch`. Some bundled `caseXXX.m` files are copied from, derived from, or modified from MATPOWER and other public test-system sources. Preserve per-file attribution notices when redistributing modified versions, and see the top-level `NOTICE` file for third-party attribution details.

The optional KLU acceleration on the bordered-solve hot path links **SuiteSparse/KLU** (Timothy A. Davis), distributed under the GNU LGPL-2.1-or-later license. The `klurf` MEX source and a cross-platform build script are provided in the top-level `klurf/` folder; see `klurf/README_KLURF.md` and the SuiteSparse license terms.

9 Citing this solver

If you use results from any release, please cite the HEBC paper:

> D. Wu and B. Wang, "Holomorphic Embedding Based Continuation Method for Identifying Multiple Power Flow Solutions," *IEEE Access*, vol. 7, pp. 86843–86853, 2019. doi: 10.1109/ACCESS.2019.2925384.

For citing the software package itself, see the top-level `CITATION.cff` file. The foundational and related works (elliptical branch tracing, Wu's 2017 UW–Madison thesis, the ACOPF extension, the voltage-stability-margin application, and the IEEE Dataport solution-set collection) are listed in each release's user guide.